Lập trình mạng · Chương 3

Bài toán ứng dụng: chat thời gian thực bằng WebSocket

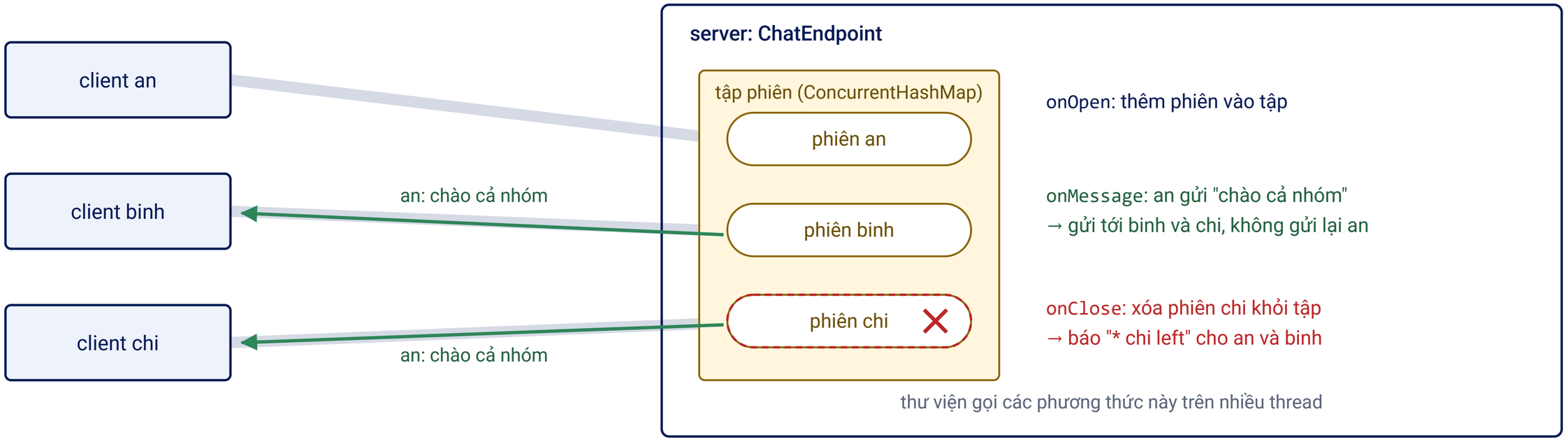
TS. Đặng Ngọc Hùng

Phòng chat: nhiều phiên cùng lúc, tin gửi tới mọi phiên còn lại

1. Server giữ **nhiều phiên WebSocket** cùng lúc; mỗi tin gửi tới **mọi phiên còn lại**
2. Phiên mở và đóng bất cứ lúc nào
3. Các phiên chạy đồng thời: tập phiên bị **nhiều thread** truy cập

> **Bài toán:** server phòng chat quản lý tập phiên đúng khi nhiều thread cùng truy cập; client dùng API của JDK.

Endpoint giữ tập phiên: onOpen thêm, onMessage gửi, onClose xóa



WebSocket endpoint: thư viện gọi `onOpen`, `onMessage`, `onClose` cho từng **phiên WebSocket**; endpoint giữ tập phiên.

ChatEndpoint: ba phương thức theo vòng đời của phiên

Java

```java
@ServerEndpoint("/chat")
private static final Map<String, Session> sessions = new ConcurrentHashMap<>();  // nhiều thread cùng truy cập
@OnOpen
public void onOpen(Session session) {
    sessions.put(session.getId(), session);
@OnMessage
public void onMessage(Session session, String text) {
    int sent = broadcast(session, name + ": " + text);
@OnClose
public void onClose(Session session, CloseReason reason) {
    sessions.remove(session.getId());              // phiên đã đóng: không gửi tới nữa
```

Kết quả chạy

```
open  an (sessions: 1)
open  binh (sessions: 2)
open  chi (sessions: 3)
an: chào cả nhóm -> sent to 2
binh: chào an -> sent to 2
close chi, code 1000 (sessions: 2)
an: chi vừa rời phòng -> sent to 1
close an, code 1000 (sessions: 1)
close binh, code 1000 (sessions: 0)
```

Gửi tới mọi phiên khác, mỗi phiên một thao tác gửi tại một thời điểm

Java

```
for (Session s : sessions.values()) {
    if (s == from || !s.isOpen()) {
        continue;
    }
    try {
        synchronized (s) {                    // mỗi phiên chỉ một thao tác gửi tại một thời điểm
            s.getBasicRemote().sendText(text);
        }
```

Kết quả chạy

```
* chi joined
an: chào cả nhóm
* chi left
an: chi vừa rời phòng
* an left
```

Tyrus chạy endpoint ở port 9401, ba client chat trong ba tiến trình

```java
Server server = new Server("localhost", 9401, "/", Map.of(), ChatEndpoint.class);
server.start();                                  // endpoint ở ws://localhost:9401/chat
```

Kết quả chạy

```
* binh joined
* chi joined
binh: chào an
* chi left
```

Tập phiên an toàn đồng thời, gửi có đồng bộ, phiên đã mất phải được đóng

1. Tập phiên bị nhiều thread truy cập: dùng **ConcurrentHashMap**, không dùng `HashMap`
2. Mỗi phiên **một thao tác gửi tại một thời điểm**: đồng bộ hóa, hoặc hàng đợi gửi riêng cho mỗi phiên
3. Client mất mạng không gửi close: đặt **thời gian chờ** cho phiên hoặc gửi **ping** định kỳ

Câu hỏi 1

Phòng chat giữ tập phiên đang mở; thư viện gọi onOpen, onMessage, onClose trên nhiều thread cùng lúc. Nên dùng cấu trúc dữ liệu nào?

A. HashMap, vì thêm và xóa đều nhanh.

B. ConcurrentHashMap, vì cho nhiều thread thêm, xóa và duyệt đồng thời an toàn.

C. ArrayList, vì chỉ cần duyệt để gửi tin.

D. HashMap gán cho một biến volatile.

Câu hỏi 1

Phòng chat giữ tập phiên đang mở; thư viện gọi onOpen, onMessage, onClose trên nhiều thread cùng lúc. Nên dùng cấu trúc dữ liệu nào?

A. HashMap, vì thêm và xóa đều nhanh.

B. ConcurrentHashMap, vì cho nhiều thread thêm, xóa và duyệt đồng thời an toàn.

C. ArrayList, vì chỉ cần duyệt để gửi tin.

D. HashMap gán cho một biến volatile.

ConcurrentHashMap cho nhiều thread đọc, ghi và duyệt đồng thời mà không hỏng cấu trúc. volatile chỉ bảo đảm các thread thấy tham chiếu mới nhất, không bảo vệ thao tác bên trong HashMap.

Câu hỏi 2

Một client mất mạng đột ngột, không gửi được frame close. Server phát hiện phiên đó đã mất bằng cách nào?

A. onClose được gọi ngay khi client mất mạng.

B. TCP báo lỗi ngay lập tức cho server.

C. Server đặt thời gian chờ tối đa cho phiên hoặc gửi ping định kỳ; quá hạn không có dữ liệu hay pong thì đóng phiên.

D. Không cần phát hiện, vì phiên tự hết hạn sau mỗi tin.

Câu hỏi 2

Một client mất mạng đột ngột, không gửi được frame close. Server phát hiện phiên đó đã mất bằng cách nào?

A. onClose được gọi ngay khi client mất mạng.

B. TCP báo lỗi ngay lập tức cho server.

C. Server đặt thời gian chờ tối đa cho phiên hoặc gửi ping định kỳ; quá hạn không có dữ liệu hay pong thì đóng phiên.

D. Không cần phát hiện, vì phiên tự hết hạn sau mỗi tin.

Khi client mất mạng mà không có gói nào báo, TCP không biết đầu kia đã mất, nên không có sự kiện nào để gọi onClose. Server phải tự phát hiện bằng thời gian chờ tối đa của phiên hoặc ping/pong.

Tổng kết

1. **WebSocket endpoint**: thư viện gọi `onOpen`, `onMessage`, `onClose` cho từng **phiên WebSocket**

2. Tập phiên trong `ConcurrentHashMap`; mỗi tin gửi tới mọi phiên khác, gửi có đồng bộ

3. Phiên đã mất: thời gian chờ hoặc **ping/pong**

Từ khóa cần nhớ WebSocket endpoint · phiên WebSocket · onMessage · ping/pong

Mở rộng

Chủ đề nâng cao

- STOMP trên WebSocket
- Chat nhiều server qua message broker (Chương 4)

Bài tập mở rộng

Thêm phòng chat theo tên: client gửi /join <phòng>, tin chỉ gửi tới các phiên cùng phòng.

Tiếp theo

HTTP/2: binary framing và multiplexing